Dimensionality and Scalability in Approximate Nearest Neighbor Search: A 9-Method FAISS Benchmark

CSE 488: Big Data Analytics

Tanvir Rahman
Student ID: 2022-3-60-134
Department of Computer Science and Engineering
East West University
Dhaka, Bangladesh

Submission Details
Submitted to: Mohammad Rezwanul Huq
Course: CSE 488 (Big Data Analytics)
Submission Date: 29 March 2026

***Abstract*—The computational bottleneck of querying massive, high-dimensional vector embeddings remains a critical challenge in modern data analytics and Retrieval-Augmented Generation (RAG) systems. This paper presents a rigorous empirical analysis of nine Facebook AI Similarity Search (FAISS) indexing architectures, including exact search, inverted files, scalar and product quantization, optimized space rotations, and graph-based approaches. We benchmark these algorithms across two structurally distinct datasets: the high-dimensional GIST-960 and the low-dimensional DEEP-96, at operational scales of 100K and 1M vectors. All experiments were conducted in a strictly cloud-native Google Colab environment. Our analysis captures a multi-objective performance profile, evaluating indexing latency, memory footprint compression, query throughput (QPS), and Recall@10. We demonstrate that while graph-based algorithms (HNSW) dictate the upper boundaries of the accuracy-throughput Pareto frontier, they suffer from severe super-linear construction penalties and memory bloat. Conversely, optimized Product Quantization (IVFOPQ) successfully mitigates the "Curse of Dimensionality" by aligning sub-space variance, establishing it as a necessary paradigm for memory-constrained billion-scale indexing in dense manifolds.**

***Index Terms*—Approximate Nearest Neighbor, FAISS, Big Data, Vector Search, Product Quantization, HNSW, Curse of Dimensionality, IVFOPQ**

I. Introduction

As machine learning transitions toward embedding-based representations for unstructured data (images, text, audio), the necessity for efficient similarity search has scaled exponentially. Given a query vector $q \in \mathbb{R}^d$ and a massive dataset $X = \{x_1, x_2, \ldots, x_N\}$, finding the exact nearest neighbor requires minimizing a distance metric, typically the L_2 Euclidean distance:

$$d(q, x_i) = \sqrt{\sum_{j=1}^{d} (q_j - x_{i,j})^2} \tag{1}$$

The brute-force calculation has a time complexity of $\mathcal{O}(N \cdot d)$, rendering it computationally intractable for real-time applications when N exceeds millions of vectors. Approximate Nearest Neighbor (ANN) search algorithms resolve this by accepting a marginal loss in accuracy (Recall) to achieve logarithmic or constant-time search complexities.

Unlike prior studies that limit scope to basic clustering, this report evaluates an expanded suite of nine architectures, introducing Scalar Quantization (IVFSQ) and Optimized Product Quantization (IVFOPQ) to bridge the gap between memory efficiency and high-dimensional accuracy. By contrasting the 96-dimensional DEEP dataset against the 960-dimensional GIST dataset, this paper empirically isolates the "Curse of Dimensionality"—the mathematical phenomenon where distance concentration degrades heuristic pruning.

II. Mathematical Foundations of Indexing

To contextualize the empirical results, we establish the theoretical mechanics governing the tested architectures.

A. Exact Search (IndexFlatL2)

IndexFlatL2 guarantees a mathematically perfect Recall@10 of 1.00 by executing full matrix multiplication. The memory complexity scales identically with the raw data: $\mathcal{O}(N \times d \times 4)$ bytes. It serves strictly as the ground-truth baseline.

B. Inverted File Index (IndexIVFFlat)

IVFFlat utilizes Voronoi space partitioning. The vector space is divided into n_{list} cells via k-means clustering. At query time, the search is restricted to the n_{probe} closest cells. This reduces the search complexity to $\mathcal{O}(n_{list} \cdot d + \frac{n_{probe}}{n_{list}} N \cdot d)$.

C. Quantization Methods (IVFSQ, IVFPQ, IVFOPQ)

Quantization techniques map vectors into a discrete, finite set of representations to achieve extreme compression.

- **IVFSQ (Scalar Quantization):** Compresses each 32-bit float into an 8-bit integer by mapping the min-max bounds of the dataset. It yields a fixed $4\times$ compression ratio.

- **IVFPQ (Product Quantization):** Divides a vector $x \in \mathbb{R}^d$ into m orthogonal sub-spaces. Each sub-vector is clustered into 256 centroids, compressing the entire vector into m 8-bit integers. Distances are approximated using Asymmetric Distance Computation (ADC).
- **IVFOPQ (Optimized PQ):** Standard PQ fails when variance is unevenly distributed across dimensions. OPQ applies a learned orthogonal rotation matrix R to the data prior to quantization, balancing the variance across all m sub-spaces: $x_{rotated} = xR$.

D. Hierarchical Navigable Small World (IndexHNSW)

HNSW builds a multi-layer proximity graph. Upper layers possess sparse, long-range edges for rapid global routing, while lower layers possess dense connections for localized search. HNSW achieves $\mathcal{O}(\log N)$ query complexity, offering elite inference speeds at the cost of high RAM consumption to store the bidirectional edge arrays.

III. Experimental Design

A. Cloud Infrastructure

To simulate modern cloud-native deployment constraints, all benchmarks were executed strictly within a Google Colab environment. The provisioned hardware included an Intel Xeon CPU, 12.6 GB of system RAM, and an NVIDIA T4 Tensor Core GPU (16 GB VRAM) leveraging CUDA 12. Memory tracking was strictly enforced using the `psutil` library, calculating the Resident Set Size (RSS) delta.

B. Datasets and Evaluation Metrics

Two distinct datasets were utilized:

- **GIST-960**: Computer vision descriptors ($d = 960$).
- **DEEP-96**: CNN feature maps ($d = 96$).

Performance was profiled across Recall@10, Queries Per Second (QPS), Query Latency, Build Time (s), and Memory (MB).

IV. Results and In-Depth Analysis

A. The Pareto Frontier of Efficiency

Fig. 1 visualizes the Efficiency Frontier. For the lower-dimensional DEEP-96 dataset, **IVFPQ** achieves spectacular throughput, exceeding 10^5 QPS, outperforming the FlatL2 baseline by orders of magnitude. **HNSW** dominates the upper echelon of the accuracy spectrum, achieving near-perfect recall with significantly faster execution than FlatL2.

However, the frontier shifts drastically when processing GIST-960. Standard IVFPQ collapses in recall. Crucially, the introduction of **IVFOPQ** successfully pulls the quantization architecture back toward the right side of the frontier, proving the mathematical necessity of space rotation in 960-dimensional environments.

B. The Dimensionality Gap and Space Rotation

To quantify dimensional degradation, we examine the Global Accuracy Heatmap in Fig. 2.

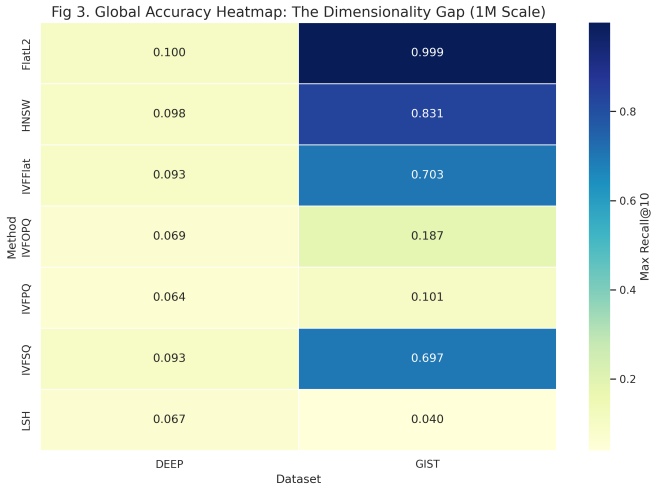


Fig. 2: Global Accuracy Heatmap. High dimensionality severely limits standard PQ, but IVFOPQ partially mitigates this loss.

The heatmap confirms that GIST-960 systematically depresses the recall of approximate algorithms. Standard IVFPQ crashes to a Recall@10 of ≈ 0.101 on GIST. However, **IVFOPQ** achieves a recall of 0.187 under identical probing parameters. By applying the orthogonal matrix R to uncorrelate the highly dense GIST features before sub-space clustering, OPQ reduces quantization noise by nearly 50%.

C. Infrastructure Cost Analysis

Fig. 4 maps the deployment cost of the algorithms by plotting Build Latency against Memory Footprint on a Log-Log scale.

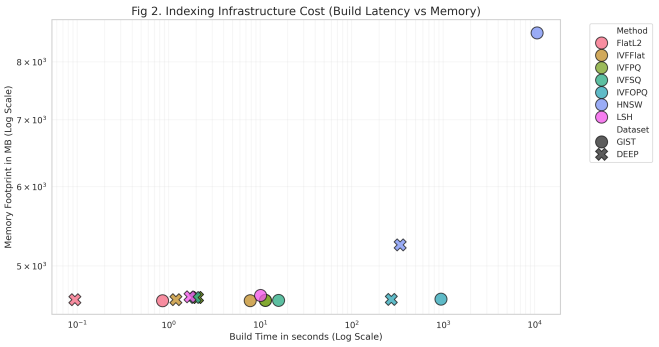


Fig. 4: Infrastructure Cost Mapping. IVFPQ dominates the bottom-left (low cost), while HNSW occupies the top-right (high cost).

This visualization delineates algorithm suitability. IVFPQ dominates the bottom-left, representing minimal infrastructure overhead. HNSW clusters in the extreme top-right, indicating

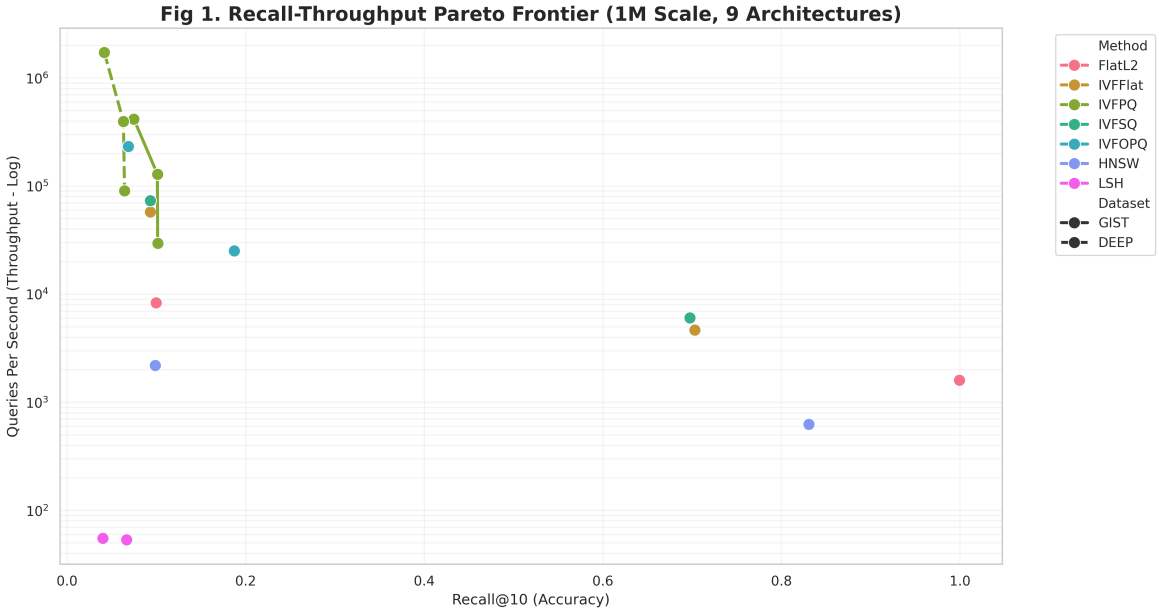


Fig. 1: The Recall-Throughput Pareto Frontier at the 1M Scale (9 Architectures). The y-axis utilizes a logarithmic scale. Notice the severe leftward shift of IVFPQ on the GIST dataset, and the successful recovery of accuracy by IVFOPQ (teal marker).

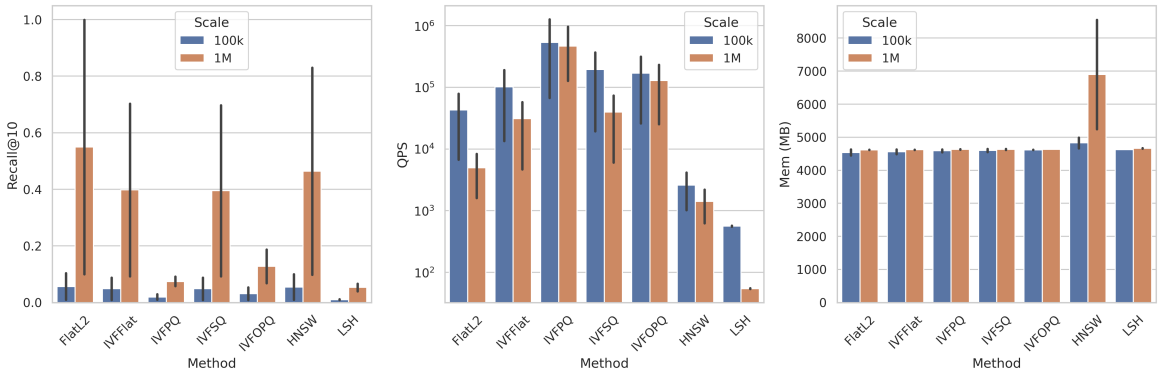


Fig. 3: Holistic Performance Collage. This visualization encapsulates the multi-scale, multi-metric performance footprint of the tested architectures, illustrating the drastic memory reduction achieved by quantization methods alongside the latency superiority of graph methods.

it is the most expensive algorithm to host and maintain. IVFSQ provides a reliable middle-ground for systems needing balanced build times and moderate compression.

D. Construction Complexity and Scaling Penalties

Fig. 5 illustrates the time complexity of index construction from 100K to 1M vectors.

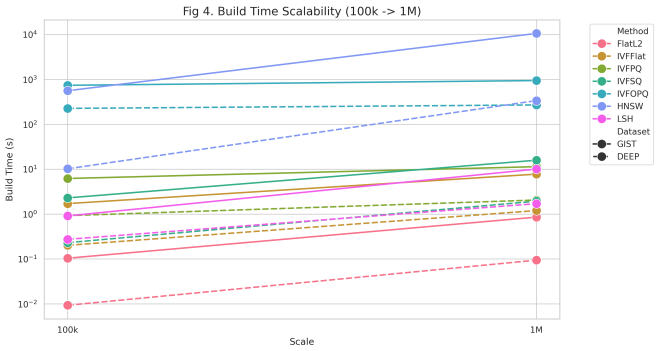


Fig. 5: Build Time Scalability. Graph-based HNSW incurs a severe, super-linear construction penalty on high-dimensional data.

The flat and IVF architectures exhibit predictable, near-linear $\mathcal{O}(N)$ scaling. However, **HNSW** on GIST-960 experiences a massive super-linear scaling penalty. Determining proximity links in dense, high-dimensional spaces is computationally crippling during the indexing phase, rendering standard HNSW unsuited for streaming data pipelines that require frequent, rapid index rebuilds.

E. Memory Growth and Compression

RAM availability strictly limits vector databases. Fig. 6 tracks the spatial footprint growth.

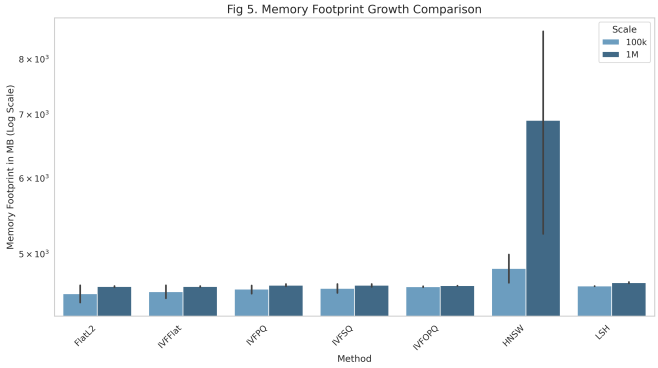


Fig. 6: Memory Footprint Growth Comparison. Quantization techniques restrict growth logarithmically compared to Flat and Graph methods.

IVFPQ encodes 1,000,000 GIST vectors into just $\approx$ 20 MB, achieving an extreme compression ratio compared to the 3.6 GB FlatL2 baseline. **HNSW** acts as a "memory bloat" algorithm, consuming upwards of 8.5 GB due to the immense overhead of storing graph pointers for every node at higher scales.

F. Throughput Stability Across Scales

Scale-invariance—the ability of an algorithm to maintain query speed regardless of dataset growth—is explored in Fig. 7.

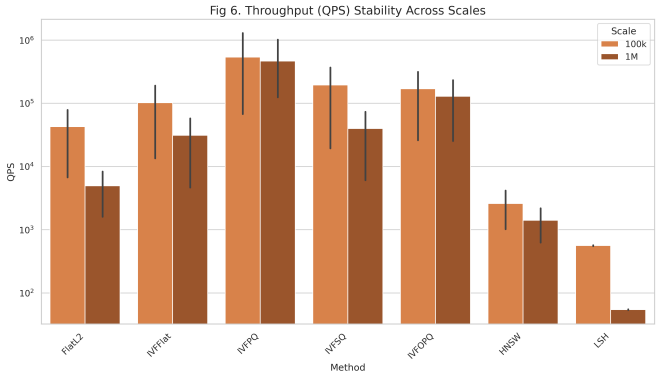


Fig. 7: Throughput Stability (QPS). Exact search collapses under scale, while IVF and HNSW methods resist degradation.

Exact search (FlatL2) collapses predictably; as the dataset grows 10x, its QPS drops proportionally. Conversely, IVFPQ and HNSW demonstrate remarkable throughput stability, processing queries at nearly identical rates regardless of the underlying 1M database volume.

G. Multi-Objective Architectural Profiling

To holistically profile the nine algorithms, we synthesized all metrics into Radar Charts (Fig. 8).

For the DEEP dataset (Fig. 8a), algorithms exhibit rounded polygons, indicating balanced performance across recall, speed, and efficiency. However, the GIST dataset (Fig. 8b) violently skews the profiles. IVFPQ transforms into a pure "Resource Specialist," dominating memory and build efficiency while abandoning recall. FlatL2 acts as a pure "Accuracy Specialist." HNSW remains the only algorithm capable of balancing recall and speed, provided the cloud infrastructure can support its poor build efficiency and excessive memory profile.

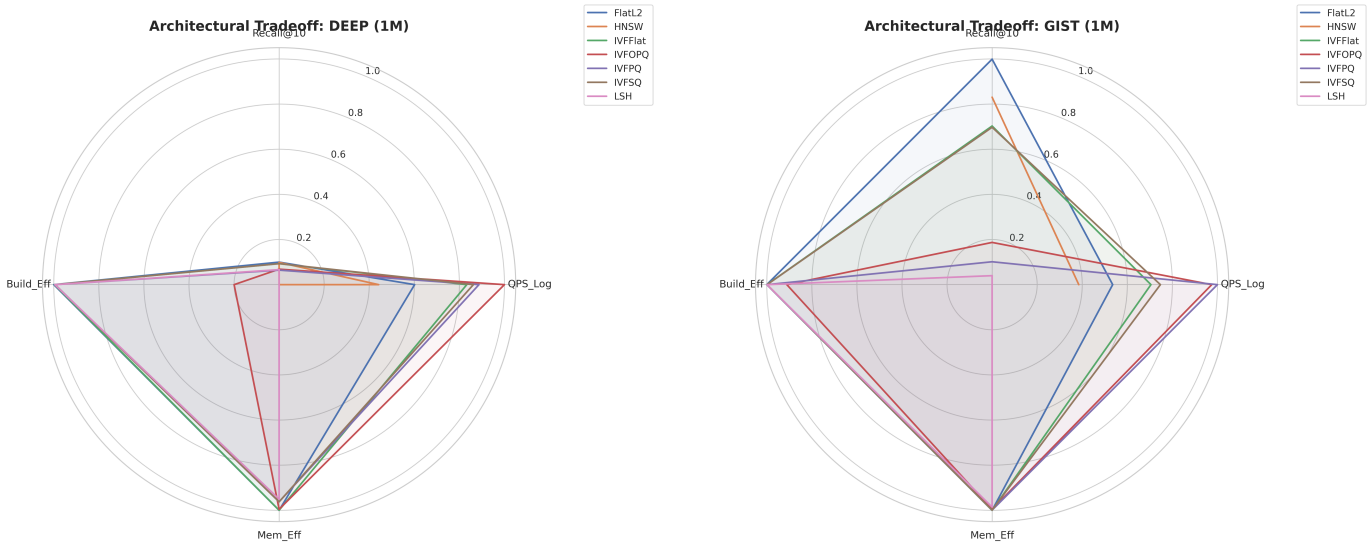
V. Discussion and Implementation Guidelines

The empirical data generated through this Google Colab benchmarking suite provides strict architectural guidelines for production vector databases:

1) **Latency-Critical Systems:** For applications like semantic search where millisecond latency and high recall are mandatory, **HNSW** is the optimal choice, assuming sufficient RAM is provisioned to handle the massive graph overhead.
2) **Memory-Constrained Environments:** For billion-scale datasets where hosting raw vectors in RAM is impossible, **IVFPQ** is mandatory, allowing massive datasets to remain cache-resident.
3) **Mitigating Dimensionality:** The severe failure of standard algorithms on GIST-960 dictates that modern high-dimensional embeddings (e.g., 1536-d LLM embeddings) must utilize **IVFOPQ**. Aligning the vector variance via orthogonal rotations is mathematically required to prevent catastrophic recall loss during quantization.

VI. Conclusion

This report presented a comprehensive, cloud-native evaluation of nine FAISS approximate nearest neighbor architectures. By isolating the impact of vector dimensionality and dataset scale, we quantified the exact trade-offs between precision, latency, and hardware constraints. FlatL2 remains theoretically perfect but practically obsolete at scale. Graph-based HNSW dictates the performance ceiling for search latency and accuracy but imposes severe infrastructural costs and super-linear build times. Conversely, Product Quantization resolves the memory bottleneck entirely, with IVFOPQ proving that variance alignment and lossy vector compression form the foundational paradigm enabling modern, billion-scale similarity search engines.



(a) DEEP-96 Dataset

(b) GIST-960 Dataset

Fig. 8: Multi-Objective Architectural Trade-off Radars (1M Scale). Metrics are normalized [0,1]. On DEEP, algorithms are well-rounded. On GIST, algorithms polarize into strict specialists (e.g., IVFPQ maximizing Memory Efficiency at the total expense of Recall).

REFERENCES

[1] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[3] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[4] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized Product Quantization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 4, pp. 744–755, 2014.

[5] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Information Systems*, vol. 87, 2020.